

| 操作系统作业5

利用`socket`编程，实现不同主机上的进程（或者同一主机上不同进程）之间的通信。发送者进程可以向接收者进程发文件或文字。接收者进程收到了文件之后，能够自动向发送者进程发送接收回执。接收回执可以是一句话。

实验环境：

1. windows主机
2. Ubuntu系统虚拟机

结果展示：

| 发送文本

客户端：

```
回执：文本已接收。  
(base) root@REFINE-LV:~# python client.py  
输入服务器IP: 192.168.0.113  
输入服务器端口: 6666  
发送文本(t)或文件(f): t  
输入要发送的文本: Hello world!  
回执：文本已接收。  
(base) root@REFINE-LV:~# |
```

服务器：

```
F:\Anaconda\envs\commonuse\python.exe D:\project\py\Android\server.py  
服务器启动，监听 0.0.0.0:6666  
来自 ('192.168.0.113', 54108) 的连接  
收到文本: Hello world!
```

| 发送文件

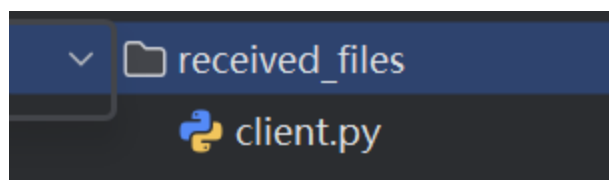
客户端：

```
(base) root@REFINE-LV:~# python client.py
输入服务器IP: 192.168.0.113
输入服务器端口: 6666
发送文本(t)或文件(f): f
输入文件路径: client.py
回执: 文件 client.py 已接收。
(base) root@REFINE-LV:~# |
```

服务器：

```
收到文本: Hello world!
来自 ('192.168.0.113', 54114) 的连接
文件 client.py 接收完毕。
```

接收到文件：



I 附录

服务器代码：

```
import socket
import struct
import json
import os

def handle_client(conn):
    try:
        # 接收头信息长度
        header_length_data = conn.recv(4)
        if len(header_length_data) != 4:
            print("选择模式无效")
            return
        header_length = struct.unpack('!I', header_length_data)[0]
        # !表示网络字节序, I表示4字节无符号整数
        # 接收头信息
        header_json = b''
```

```

while len(header_json) < header_length:
    chunk = conn.recv(header_length - len(header_json))
    if not chunk:
        raise Exception("Header接收不全")
    header_json += chunk
header = json.loads(header_json.decode('utf-8'))
#解码后的 header 是一个字典, 例如:
# json
# {"type": "text", "size": 13}           // 文本
# {"type": "file", "name": "test.txt", "size": 1024} // 文件
data_type = header['type']

if data_type == 'text':
    # 接收文本
    text_size = header['size']
    text_data = b''
    remaining = text_size
    while remaining > 0:
        chunk = conn.recv(min(4096, remaining))
        if not chunk:
            raise Exception("文本接收不全")
        text_data += chunk
        remaining -= len(chunk)
    print(f"收到文本: {text_data.decode('utf-8')}")
    receipt = "文本已接收。"
    conn.send(receipt.encode('utf-8'))
elif data_type == 'file':
    # 接收文件
    file_name = header['name']
    file_size = header['size']
    save_dir = 'received_files'
    os.makedirs(save_dir, exist_ok=True)
    save_path = os.path.join(save_dir, file_name)

    received = 0
    with open(save_path, 'wb') as f:
        while received < file_size:
            chunk = conn.recv(min(4096, file_size - received))
            if not chunk:
                raise Exception("文件接收不全")
            f.write(chunk)
            received += len(chunk)
    print(f"文件 {file_name} 接收完毕。")
    receipt = f"文件 {file_name} 已接收。"
    conn.send(receipt.encode('utf-8'))
else:

```

```

        print("未知数据类型。")
    except Exception as e:
        print(f"处理错误: {e}")
    finally:
        conn.close()

def main():
    host = '0.0.0.0'
    port = 6666
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.bind((host, port))
    s.listen(5) # 开始监听, 允许最多5个待处理连接
    print(f"服务器启动, 监听 {host}:{port}")

    try:
        while True:
            conn, addr = s.accept()# conn: 一个新的 socket 对象, 专门用于与当前连接
            的客户端通信。
            print(f"来自 {addr} 的连接")
            handle_client(conn)
    except KeyboardInterrupt:
        print("服务器关闭。")
    finally:
        s.close()

if __name__ == '__main__':
    main()

```

客户端代码:

```

import socket
import struct
import json
import os

def send_text(sock, text):
    text_bytes = text.encode('utf-8')
    header = {
        'type': 'text',
        'size': len(text_bytes)
    }
    header_json = json.dumps(header).encode('utf-8')
    header_length = struct.pack('!I', len(header_json))
    sock.sendall(header_length)
    sock.sendall(header_json)

```

```

sock.sendall(text_bytes)

def send_file(sock, file_path):
    file_name = os.path.basename(file_path)
    file_size = os.path.getsize(file_path)
    header = {
        'type': 'file',
        'name': file_name,
        'size': file_size
    }
    header_json = json.dumps(header).encode('utf-8')
    header_length = struct.pack('!I', len(header_json))
    sock.sendall(header_length)
    sock.sendall(header_json)

    with open(file_path, 'rb') as f:
        while True:
            data = f.read(4096)
            if not data:
                break
            sock.sendall(data)

def main():
    server_ip = input("输入服务器IP: ")
    server_port = int(input("输入服务器端口: "))
    mode = input("发送文本(t)或文件(f): ").lower()

    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        s.connect((server_ip, server_port))
        if mode == 't':
            text = input("输入要发送的文本: ")
            send_text(s, text)
        elif mode == 'f':
            file_path = input("输入文件路径: ")
            if not os.path.isfile(file_path):
                print("文件不存在。")
                return
            send_file(s, file_path)
        else:
            print("无效模式。")
            return

    # 接收回执
    receipt = s.recv(1024).decode('utf-8')
    print("回执:", receipt)

```

```
except Exception as e:
    print(f"错误: {e}")
finally:
    s.close()

if __name__ == '__main__':
    main()
```